

```
plt.figure(figsize = (20,4))
plt.plot(tone_1)
```

```
[<matplotlib.lines.Line2D at 0x7f72549c2908>]
```

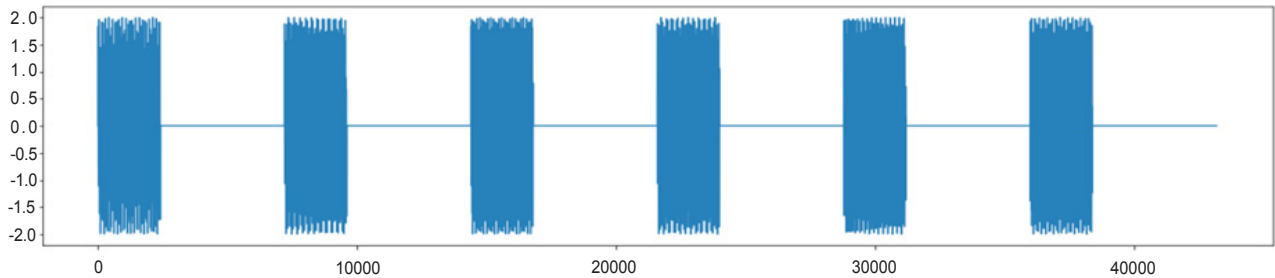


Fig. 2: Sound frequencies with a customisable speed

```
[<matplotlib.lines.Line2D at 0x7f7254407630>]
```

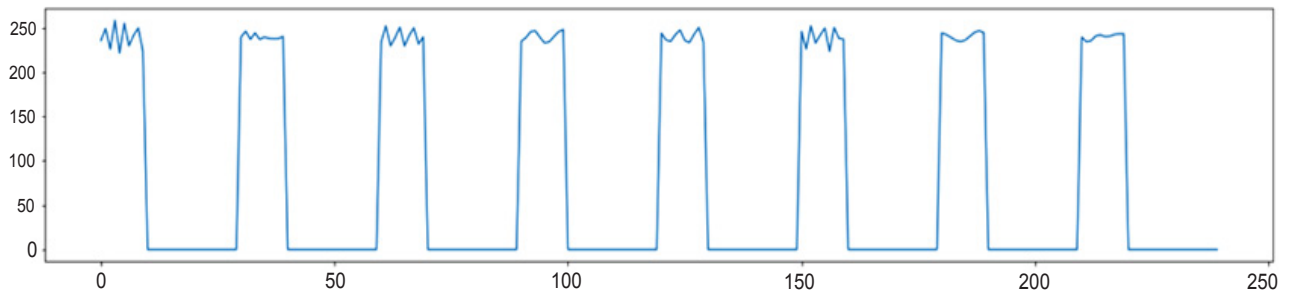


Fig. 3: DTMF frequency transformation

special character series of 1498*0:

```
tone_1=dtmf_encoder('1498*0')
IPython.display.Audio(tone_1, rate=Fs)
```

The above command will also display the audio representations of the frequencies with a customisable speed. We can see their size and represent this in a graphical form, as shown in Fig. 2.

Similarly, we may also decode the DTMF frequencies into required digits as before using a simple algorithm where we try to dissect the numbers according to the frequency ranges they represent. We check the upper bound and lower bound

DTMF SIGNAL

Digits	Upper bound	Lower bound
1	1209	697
2	1336	697
3	1477	697
4	1209	770
5	1336	770
6	1477	770
7	1209	852
8	1336	852
9	1477	852
*	1209	941
0	1336	941
#	1477	941

of the frequencies by converting it into NumPy arrays. For example, to decode '1498' we get a frequency between 697Hz to 1209Hz, so we know that the first digit is one. The next one lies between 770Hz to 1209Hz, and so on.

Hence, we execute the decoding code in the `dtmf_decode()` function by calling the functions given below:

```
print(decode_dtmf (tone_2)) # works perfectly!!
['7', '6', '5', '8', '9', '4', '1', '0']
print(decode_dtmf(tone_1))
['1', '4', '9', '8', '*', '0']
```

Now, we can see how `tone_1` and `tone_2`, which were an overlapping sinusoidal wave of frequency '2', are being converted to the original sequence with minimal error.

Fig. 3 shows the energy graph of DTMF frequency transformation.

DTMF is used even today. Often, when dialling a customer care number, we are greeted with the auto-

mated response of 'For English press 1, for Hindi press 2', and so on. This selection menu on voice calls would have been a challenge were it not for DTMF. The key that the user taps for a particular selection is encoded as DTMF and then decoded at the receiver's end to get the user's choice.

However, the use of DTMF was not as simple as it seems. It was vulnerable to an attack called 'phreaking.' This attack used a malicious actor to send unauthorised DTMF signals by hooking up some custom hardware on the telephone lines. In some regions of the United States, it is illegal to connect anything to the telephone line for the same reason.

A lot of applications and systems can be built using the concepts explained in this article. So this retro technology is evergreen! **EFY**

This article was first published in October 2022 issue of Open Source For You magazine.